

UniNotes

COMPUTER SCIENCE

Data Structures Summary Notes

Arrays, stacks, queues, linked lists, trees and algorithmic trade-offs explained clearly.

How to use these notes

Start with the key definitions, then use the tables/diagrams to connect ideas. Finish with the exam checklist. The aim is high information density: short wording, clear concepts, no filler.

1. Why data structures matter

A data structure organises data so operations such as access, search, insert and delete can be performed efficiently. The "best" structure depends on the operations you need most.

Array	Indexed access $O(1)$	Insert/delete in middle can be $O(n)$.
Linked list	Insert/delete with known node $O(1)$	Access by position is $O(n)$.
Stack	Push/pop $O(1)$	Only top item directly accessible.
Queue	Enqueue/dequeue $O(1)$	FIFO access only.
Hash table	Average lookup $O(1)$	Collisions; no natural order.
BST	Average search/insert $O(\log n)$	Can degrade to $O(n)$ if unbalanced.

2. Stack vs queue



- Stack = **LIFO** (last in, first out). Uses: function call stack, undo, expression parsing.
- Queue = **FIFO** (first in, first out). Uses: print queues, task scheduling, breadth-first search.

3. Linked lists

Each node stores data plus a link to another node. Singly linked lists point forward; doubly linked lists point both forward and backward.

Insert/delete near a known node is cheap because links are changed rather than shifting many elements. The trade-off is extra pointer memory and slow random access.

4. Trees and traversal

A tree models hierarchical data. In a binary tree each node has at most two children. In a binary search tree (BST), values smaller than a node go left and larger values go right.

Pre-order	root-left-right	Copy/serialise a tree.
In-order	left-root-right	Sorted output from a BST.
Post-order	left-right-root	Delete/free a tree.
Level-order	breadth by depth	Shortest-layer exploration.

5. Big-O essentials

$O(1)$	constant	array index lookup
$O(\log n)$	grows slowly	balanced BST search
$O(n)$	linear	scan list
$O(n \log n)$	efficient sorting class	merge sort
$O(n^2)$	nested growth	simple double loop

Exam tip

State the operation as well as the structure. Saying "arrays are $O(1)$ " is incomplete; indexed access is $O(1)$, while searching an unsorted array is $O(n)$.

Exam-ready checklist

- Explain LIFO and FIFO with one example each.
- Compare array and linked-list insertion/access.
- Know the four standard tree traversals.
- Interpret $O(1)$, $O(\log n)$, $O(n)$ and $O(n^2)$.
- Choose a data structure based on required operations, not popularity.

Fast recall

If you cannot explain a concept in one or two sentences without looking, revisit that section. Aim to recall the definition, mechanism and one example.

Uni Notes keeps revision focused: concise explanations, useful detail, and diagrams only where they improve understanding.